程序设计基础及语言 —— C++ 大学教程

第 2 章 C++ 编程入门、输入/输出和运算符

邱 远

东南大学 网络空间安全学院
yuanqiu@seu.edu.cn

2026 年 10 月

目录

输出文本

整数相加

比较大小

程序一：输出一行文本

```cpp
/**
 * @file fig02_01.cpp
 * Text-printing program.
 */
#include <iostream> // allows program to output data to the screen

// function main begins program execution
int main() {
    std::cout << "Welcome to C++!\n"; // display message
    return 0; // indicate that program ended successfully
} // end function main
```

```
Welcome to C++!
```

Line 1-4：注释

```
1  /**
2   * @file fig02_01.cpp
3   * Text-printing program.
4   */
5  #include <iostream> // allows program to output data to the screen
6
7  // function main begins program execution
8  int main() {
```

代码的 1-4、7 行为注释（Comment）。注释不影响编译和运行，被编译器直接忽略，主要用来说明程序目的、作者、日期等，帮助人们理解。C++ 的注释有两种格式：

- 短注释：双斜线 (//) 会将这行之后的所有内容注释掉（第 5、7 行）
- 长注释：一对/* 与 */会将之间的内容注释掉（第 1-4 行）

- 仅长注释可跨行。使用短注释时，需要在每行打//
- 严格来说，`int/*eger*/ main`这样的注释是合法的，但很少见

Line 1-4：注释（拓展）

- 程序员最讨厌的两件事：1. 写注释；2. 看没有注释的代码
- 注释不仅是给别人看，也是给以后的自己看
- 至少在每个文件的开头要有一个注释，来说明这个文件做了什么
- doxygen 可以直接从规范注释的 C++ 代码生成网页版文档
- 大部分编辑器中，将当前行或选定区域注释/取消的快捷键是 Ctrl+/

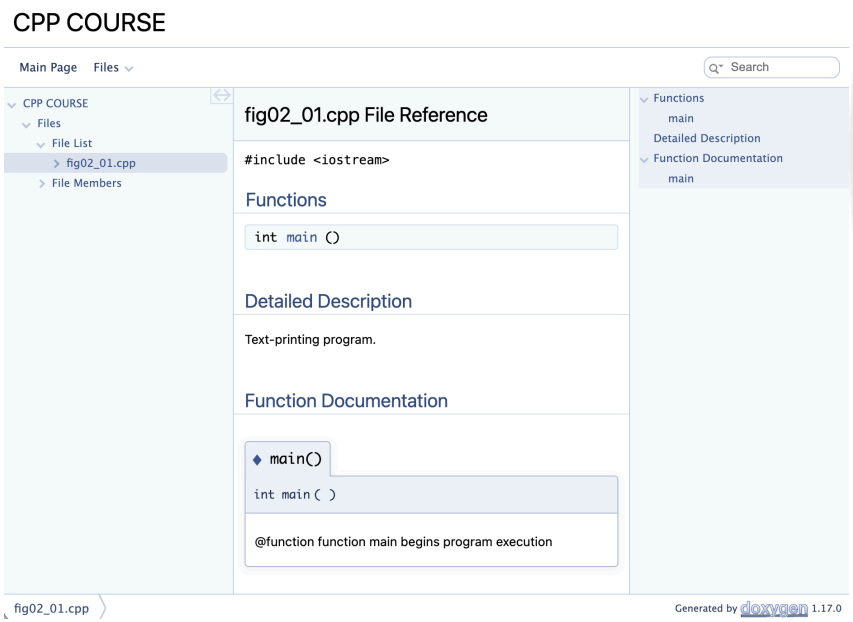


图 1: 由 doxygen 生成的网页版文档

Line 5： #include 预处理指令

```
5  #include <iostream> // allows program to output data to the screen
```

- 第 5 行的#include是一个预处理指令
- 预处理 (#) 在编译之前执行。可以认为此处是将这一行完全替换为了 <iostream> 这个头文件的内容
- 直观地来理解，如果有以下三个文件，那程序能正常编译运行

```
1  // top.half
2  int main() {
```

```
1  // bottom.half
2  std::cout << "Welcome to C++!\n";
3      return 0;
4  }
```

```
1  // example.cpp
2  #include <iostream>
3  #include "top.half"
4  #include "bottom.half"
```

只是示意，千万不要这么写代码！这也是 C++ 高自由度的体现。

Line 5： iostream

https://cplusplus.com/reference/iostream/

- 任何一个 (正确的) 程序都一定有输出
- 只要涉及输出到屏幕（或从键盘读取输入) 就需要引用这个头文件
- 我们在后续章节再介绍什么是头文件。现在暂且认为头文件是一个“词典”，里面有一些我们需要用到的“词”（函数）
- 虽然编译器不会严格检查，但习惯上，“官方”的头文件用`#include <>`来引用，自己写的头文件用`#include ""`来引用
- 在很长一段时间里（直到第 14 章），你代码中（注释后的）第一行都会是`#include <iostream>`

可以尝试去掉这一行，看看报错信息是什么。

Line 6：空行与空白

```
5  #include <iostream> // allows program to output data to the screen
6
7  // function main begins program execution
8  int main() {
```

空行与空白主要增加程序的阅读性，编译器不会在意它们。

- 除了字母与字母之间的空格会改变含义，C++ 对于符号与符号间、符号与字母间的空格非常宽松。例如以下写法是一致的
  - `int main(){`
  - `int main ( ) {`
- 类似地，空行和换行也并非必须。理论上 main 函数可以用一行写完，但实际开发时应当确保可读性
  - `int main(){std::cout<<"Welcome...\n";return 0;}`

Line 8：main 函数

https://en.cppreference.com/cpp/language/main_function

```
8  int main() { ... }
```

函数 (Function)

- 数学中的函数：对于给定输入 x，输出对应值 $f(x)$
- 计算机中的函数：
  - (function) 对于给定输入 x，输出“返回值” $f(x)$
  - (procedure) 对于给定输入 x，执行相应操作

C++ 函数

```
返回类型 函数名(输入参数) { // 此行称为函数头
    函数体;
    return 返回值;
}
```

“main” 是一个返回类型为`int`，没有输入参数，返回值为 0 的函数。

C++ 函数

https://cplusplus.com/doc/tutorial/functions/

```
8  int main() { ... }
```

- `int`是整型 (integer) 的关键字 (keyword)，我们下一节再展开
- 虽然 main 可以没有入参，但函数定义里的圆括号 () 不能少
- 大括号 {} 括起的部分是函数体
- 一个程序可能有很多函数。在 C++ 中，main 函数是主入口。程序从 main 的第一行开始执行，直到从 main 返回 (return) 结束
- main 的返回值是个整数，用于判断程序以何种方式结束。返回值`return 0`表示程序正常结束

Line 9：输出语句

https://en.cppreference.com/cpp/io/cout

```
9     std::cout << "Welcome to C++!\n"; // display message
```

- 在 C++ 中，一条语句 (statement) 以分号结尾，分号不可省
  - 预处理指令没有分号，比如`#include <iostream>`
- 语句可以有很多功能，如赋值、调用函数等，这条输出语句本质上是调用了`operator<<`这个函数
- `std`是一个命名空间 (namespace)，是为了避免重名发生，暂时将`std::cout`理解为 std 这个范围内叫 cout 的家伙
- `std::cout`是标准输出流对象，搭配流插入运算符`<<`使用时，作用是将后面的信息插入到输出流中
- 双引号`" "`括起的部分称为字符串 (string)，其中的字母称为字符 (character)
- 大部分时候，一个字母/数字/符号为一个字符。但出现反斜杠 (\) 时，需要将它和紧跟着的字母符号作为一个整体看待。反斜杠是 C++ 中的转义字符 (escape character)

Line 9：转义序列

表 1: C++ 中常见的转义序列

转义	全称	含义
\n	Newline	换行符
\t	Tab	制表符
\b	Backspace	退格符
\r	Return	回车符（退回行首不换行）
\a	Alert	发出蜂鸣
\\	\	反斜线
\’	’	单引号
\”	”	双引号

Line 10：返回语句

```
10      return 0; // indicate that program ended successfully
```

- 如同前文所述，`return 0;`表示程序正常返回
- 根据 C++ 标准，main 函数执行到最后的大括号也认为成功执行，因此 main 最后的这行是可以省略的。不过并非所有情况都能省略，因此建议还是习惯写这个 return
- 我们见到了第二个关键字 “return”。关键字又称保留字，是 C++ 中比较特殊的单词，不能用来做标识符
- 关键字在编译器中会高亮

总结：用 C++ 输出文本

```cpp
/**
 * @file fig02_01.cpp
 * Text-printing program.
 */
#include <iostream> // allows program to output data to the screen

// function main begins program execution
int main() {
    std::cout << "Welcome to C++!\n"; // display message
    return 0; // indicate that program ended successfully
} // end function main
```

经过学习，我们掌握了如何用 C++ 输出文本。同时，我们也学习了注释、预处理指令、空行、main 函数等知识。

练习：用 C++ 输出文本

小练习

请尝试用 C++ 写程序输出自己的学号和一卡通号（用 \t 分隔）。

练习：用 C++ 输出文本

小练习

请尝试用 C++ 写程序输出自己的学号和一卡通号（用 \t 分隔）。

```cpp
// example.cpp
#include <iostream>
int main() {
    std::cout << "Student ID: A1234\tCard ID: 56789\n";
} // end function main
```

实际上，<<运算符是可以串联 (cascading) 的[1]。因此，也可以写成

```cpp
// example.cpp
#include <iostream>
int main() {
    std::cout << "Student ID: " << "A1234" << "\t"
              << "Card ID: " << "56789" << "\n";
} // end function main
```

这样可读性更好。

[1] 原理我们以后会讲到，目前先记住

目录

输出文本

整数相加

比较大小

程序二：两个整数相加

```cpp
/**
 * @file fig02_05.cpp
 * Addition program that displays the sum of two integers.
 */
#include <iostream> // allows program to perform input and output

int main() {
    // variable declarations
    int num1 = 0; // first integer to add (init to 0)
    int num2 = 0; // second integer to add (init to 0)
    int sum = 0; // sum of num1 and num2 (init to 0)

    std::cout << "Enter first integer: "; // prompt user for data
    std::cin >> num1; // read first integer into num1
    std::cout << "Enter second integer: ";
    std::cin >> num2; // read second integer into num2
    sum = num1 + num2; // add the numbers; store result in sum

    std::cout << "Sum is " << sum << std::endl; // display sum; end line
} // end function main
```

Line 9-11： 声明变量

```
7  int main() {
8      // variable declarations
9      int num1 = 0; // first integer to add (init to 0)
10     int num2 = 0; // second integer to add (init to 0)
11     int sum = 0; // sum of num1 and num2 (init to 0)
```

变量 (Variable)

变量是用于存储（可变）值或计算机结果的空间。

在 C++ 中声明变量 (Variable Declaration)

数据类型 变量名，变量名；

数据类型 变量名 = 初始值，变量名 = 初始值；

基本数据类型

https://en.cppreference.com/cpp/language/types

表 2: C++ 的基本数据类型

类型	关键字	字节数	取值范围
布尔型	`bool`	1	{`false`, `true`} 等价于 $\{0, 1\}$
字符型	`char`	1	-128 至 127 或 0 至 255[2]
短整型	`short`	≥ 2	-32768 至 32767
整型	`int`	≥ 2，常为 4	-2147483648 至 2147483647 (32 位)
长整型	`long`	≥ 4，常为 4 或 8	$\pm 9.22 \times 10^{18}$ (64 位)
浮点型	`float`	4	IEEE 754 标准[3]
双精度	`double`	8	IEEE 754 标准[4]

- C++ 的默认数据类型都带符号，换言之`int`就是`signed int`
- 使用`unsigned int`等声明不带符号的整型，将正范围提升至两倍

[2]取决于编译器

[3]1 位符号，8 位指数 ($10^{\pm 18}$)，小数点后 23 位

[4]1 位符号，11 位指数 ($10^{\pm 308}$)，小数点后 52 位

基本数据类型

https://en.cppreference.com/cpp/language/types

可以用下面这段代码来检查你机器上每个类型的字节数和取值范围：

```cpp
// check_size.cpp
#include <iostream>
#include <limits>

template<typename T>
void show() {
    std::cout << typeid(T).name() << '\t' << sizeof(T) << '\t'
              << std::numeric_limits<T>::min() << '\t'
              << std::numeric_limits<T>::max() << std::endl;
}

int main() {
    std::cout << "Type\tBytes\tMin\tMax" << std::endl;
    show<bool>();
    show<short>(); show<unsigned short>();
    show<int>(); show<unsigned int>();
    show<long>(); show<unsigned long>();
    show<long long>(); show<unsigned long long>();
}
```

标识符

https://en.cppreference.com/cpp/language/identifiers

变量名必须是合法的标识符 (identifier)，有如下要求

- 由字母 (a-z, A-Z)、数字 (0-9)、下划线 (_) 组成，区分大小写
- 不能用数字开头，少用下划线开头 (编译器内部，如`__int128`)
- 不能和保留关键字重复，不建议过长 (31 字符以内)
- 标识符原则上不使用拼音 (xxxx vs. school info, lv vs. green)

下划线命名法

全部小写，使用下划线连接，例如`int a_local_variable;`这种命名方式是 Google 开源项目风格[a]推荐的。

[a] https://github.com/zh-google-styleguide/zh-google-styleguide

驼峰命名法

首个词小写，后续首字母大写，例如`int aLocalVariable;`比较常见于 Java 项目中，优点是变量名较短。

变量声明语句 (拓展)

- 一个变量在使用前必须声明，并且声明后类型就无法改变
  - 好的编程习惯是在最接近第一次使用的地方声明
- 虽然std::里的函数名不是保留字，但是通常也不用作变量名

```
#include <iostream>
namespace MY {
    int cout = 2; // a variable named "cout" defined in MY namespace
}
int main() {
    int cout = 1; // a local variable named "cout"
    std::cout << cout << MY::cout; // three different "cout"s
}
```

- 变量定义有可见范围和生命周期，我们在后续章节中介绍

常量 (Constant)

在程序中永不改变的值可以声明为常量。C++ 中有 3 种声明方式：

- `#define PI 3.14`
- `const float PI_ = 3.14f;`
- `constexpr double PI__ = 3.14;`

Line 14,16：输入语句

https://en.cppreference.com/cpp/io/cin

```
9     int num1 = 0; // first integer to add (init to 0)
10    int num2 = 0; // second integer to add (init to 0)
11    int sum = 0; // sum of num1 and num2 (init to 0)
12
13    std::cout << "Enter first integer: "; // prompt user for data
14    std::cin >> num1; // read first integer into num1
15    std::cout << "Enter second integer: ";
16    std::cin >> num2; // read second integer into num2
```

- 类似于std::cout <<是写“出去”，std::cin >>理解为读“进来”
- 类似于operator<<可以串联，operator>>同样可以串联，如

```
std::cin >> num1 >> num2;
```

- 从键盘敲回车后，程序会从已输入的内容中依次提取

Line 17：赋值语句

https://cplusplus.com/doc/tutorial/operators/

```
13      std::cout << "Enter first integer: "; // prompt user for data
14      std::cin >> num1; // read first integer into num1
15      std::cout << "Enter second integer: ";
16      std::cin >> num2; // read second integer into num2
17      sum = num1 + num2; // add the numbers; store result in sum
```

等号 (=) 和加号 (+) 都是 C++ 中的运算符 (Operator)。其中 = 是赋值运算符，+ 是一个算术运算符。它们都是二元运算符。

赋值语句 (Assignment)

变量名 = 表达式;

- 由于变量名在左，有时也称左值；表达式在右，有时也称右值
- 注意和数学中的“等于”区分，赋值有方向，理解为 := 或 ←

Line 17：算术运算符

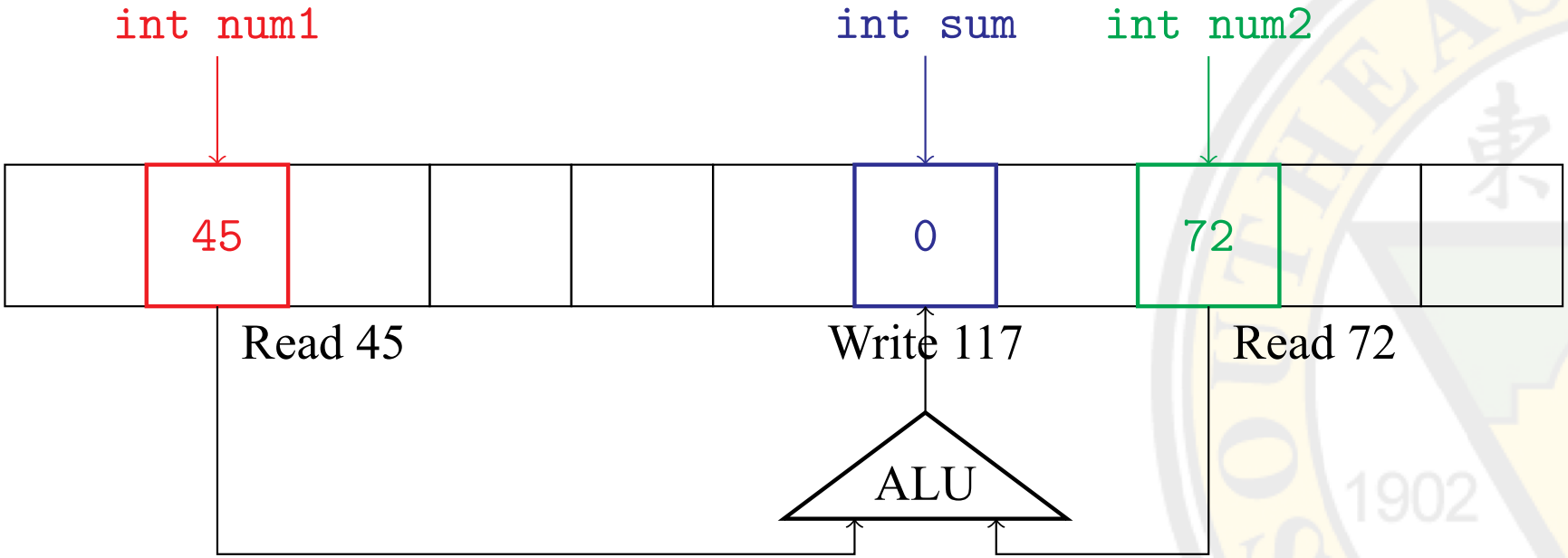
https://cplusplus.com/doc/tutorial/operators/

```
13      std::cout << "Enter first integer: "; // prompt user for data
14      std::cin >> num1; // read first integer into num1
15      std::cout << "Enter second integer: ";
16      std::cin >> num2; // read second integer into num2
17      sum = num1 + num2; // add the numbers; store result in sum
```

- C++ 的算术 (Arithmetic) 运算符：加 (+) 减 (-) 乘 (*) 除 (/) 求模 (%)
- 整数之间的除法是“整除”，例如 $5/2$ 结果为 2
  - 关于负数：$(-3)/2$ 得 -1; $(-3)\%2$ 得 -1
- 乘除模优先于加减，同优先级从左到右，用圆括号改变优先级，如 $((1+2)*3+4)*5$
- 二元运算符书写时一般左右留空格

内存的概念

每一个变量对应不同空间大小的内存。可以把内存想象成一排连续的房子，每个地址可能是空的，或住着一个变量。



- 内存初始状态不一定为 0，因此建议定义变量时都做初始化

```
#include <iostream>
int main() { int num; std::cout << num; }
```

- 由于先读取再写入，可以进行`num = num + 1;`[5]

[5]该运算称为自增

Line 19：std::endl 与流

https://en.cppreference.com/cpp/io/manip/endl

```
19      std::cout << "Sum is " << sum << std::endl; // display sum; end line
```

- std::endl有两个作用，一是输出换行符 (同'\n')，二是刷新显示流 (同std::flush)
- 我们提过**流输入**运算符>>，**流输出**运算符<<，标准**输入流** std::cin，标准**输出流** std::cout，但流是什么？
- 流 (Stream) 是有序传输的信息（字节序列）
  - 输入时，信息从键盘、鼠标等流向内存
  - 输出时，信息从内存流向显示器、打印机、磁盘、网络等
- C++ 提供了抽象的输入/输出流库，程序员只需要跟流进行交互，而无需关心不同系统/硬件的具体实现
  - 输入流：std::cin，std::ifstream，std::stringbuf
  - 输出流：std::cout，std::cerr，std::clog，std::ofstream
- std::cout有缓冲区，对于较大的程序，不刷新可能导致输出延迟

总结：用 C++ 做算术

- 变量与常量（数据类型 + 变量名）、内存的概念
  - 基本数据类型：整数型 (`bool`, `char`, `short`, `int`, `long`)、浮点型 (`float`, `double`)
  - 变量名：合法标识符（字母、数字、下划线）
- 输入语句（`std::cin >>`）、流
- 赋值语句（`y = x;`）
- 算术运算（+, -, *, /, %）及其优先级（圆括号）

练习：用 C++ 做算术

小练习

请尝试用 C++ 写程序，输入一元二次函数

$$y = ax^2 + bx + c$$

的二次、一次、常数项系数，及横坐标一点 x，输出对应函数值 y。

练习：用 C++ 做算术

小练习

请尝试用 C++ 写程序，输入一元二次函数

$$y = ax^2 + bx + c$$

的二次、一次、常数项系数，及横坐标 x，输出对应函数值 y。

```cpp
// example.cpp
#include <iostream>
int main() {
    std::cout << "Enter a, b, c, x in order for y=ax^2+bx+c: ";
    float a, b, c, x;
    std::cin >> a >> b >> c >> x;
    std::cout << "y = " << a * x * x + b * x + c << std::endl;
} // end function main
```

目录

输出文本

整数相加

比较大小

程序三：比较两数大小

```cpp
/**
 * @file fig02_13.cpp
 * Comparing ints using if statements, relational ops and equality ops
 */
#include <iostream> // allows program to perform input and output

using std::cout; // program uses cout
using std::cin; // program uses cin
using std::endl; // program uses endl

int main() {
    int num1 = 0, num2 = 0; // two integers to compare (both init to 0)
    cout << "Enter two integers to compare: "; // prompt user for data
    cin >> num1 >> num2; // read two integers

    if (num1 == num2) cout << num1 << " == " << num2 << endl;
    if (num1 != num2) cout << num1 << " != " << num2 << endl;
    if (num1 < num2) cout << num1 << " < " << num2 << endl;
    if (num1 > num2) cout << num1 << " > " << num2 << endl;
    if (num1 <= num2) cout << num1 << " <= " << num2 << endl;
    if (num1 >= num2) cout << num1 << " >= " << num2 << endl;
} // end function main
```

Line 7-9：using 声明

https://en.cppreference.com/cpp/keyword/using

```
7       using std::cout;
8       using std::cin;
9       using std::endl;
```

- using关键字用于声明命名空间中的成员，不用每次写前缀
- using namespace std;可以用于声明 std 中的所有名称
  - 小程序可用，大程序一般反对使用这句[6]，应保持“最小引用”原则
- 理论上using的位置限制很小，可以在函数中出现，更为常见的是在main之前
- 除此以外，using还有些其他作用，我们后面会介绍
  - C++11 起，可以用using stu_id_type = int;来给类型起别名

[6] std 的 string 性能不好，许多项目都会重新实现 string，直接全部using可能会起冲突

Line 16：if 语句

https://en.cppreference.com/cpp/language/if

```
16      if (num1 == num2) cout << num1 << " == " << num2 << endl;
```

选择语句（章节 4.5）

```
if (布尔表达式) {
    语句;
} // else { ... }
```

- 当布尔表达式为true时，执行大括号内的语句；否则跳过大括号执行后续语句。
- 布尔表达式外的小括号不能省略，如if (true)
- 语句应相对 if 缩进；若语句只有一句，则大括号可省略
  - 按照编程规范，即使只有一句，大括号也不应省略
  - 是否有空格空行依旧不影响编译运行
- 若表达式非布尔型，会被转为bool，此时非 0 都为true

Line 16-21：关系比较运算符

https://cplusplus.com/doc/tutorial/operators/

```
16      if (num1 == num2) cout << num1 << " == " << num2 << endl;
17      if (num1 != num2) cout << num1 << " != " << num2 << endl;
18      if (num1 < num2) cout << num1 << " < " << num2 << endl;
19      if (num1 > num2) cout << num1 << " > " << num2 << endl;
20      if (num1 <= num2) cout << num1 << " <= " << num2 << endl;
21      if (num1 >= num2) cout << num1 << " >= " << num2 << endl;
```

- C++ 共有六个关系 (Relational) 比较 (Comparison) 运算符
  - 大于 (>) 小于 (<) 大于等于 (>=) 小于等于 (<=)
  - 等于 (==) 不等于 (!=)[7]
- 关系比较运算符计算的结果是布尔型`bool`
- 注意区分赋值 (=) 与相等 (==)
- 优先级：算术运算 > 流输入输出 > 比较运算 > 等于/不等于
  - `if (a + 1 > b * 2)`
- 结合律：算术运算、流输入输出、关系比较均为同优先级从左向右
  - `2 == 1 == 0`结果为`true`，`0 == 1 < 0`结果为`true`

[7] 感叹号 (!) 同时是 C++ 的逻辑否定运算，`1 != 2`和`!(1 == 2)`结果同为`true`

赋值运算与流输入输出的返回值（拓展）

我们已经学习了：

- 算术运算 $(+, -, *, /, \%)$ 的返回值是数值 (int, float 等)
- 关系比较运算 (>,>=,<,<=,==,!=) 的返回值是布尔 (bool)

实际上，赋值运算和流运算符返回的都是他们左侧的变量，其中

- 赋值运算 (=) 为右结合，因此对于 `x = y = 2;` 是先执行`y = 2;`然后返回`y`。第二步执行的相当于是`x = y;`
- 流输入 (>>) 输出 (<<) 为左结合，因此对于 `std::cout << x << y;` 是先执行`std::cout << x;`然后返回 `std::cout`。第二步执行的相当于是`std::cout << y;`

目前学到的四类运算符，仅赋值 (=) 为右结合，其余均为左结合。优先级记忆如下：

- 算术 > 流输入输出：`std::cout << 2 * 2 + 2;`不需要括号
- 流输入输出 > 关系比较：`std::cout << (2 == 1);`需要括号
- 关系比较 > 赋值：`bool is_even = x % 2 == 0;`不需要括号

总结：用 C++ 做比较

- using 声明
  - `using std::cout;`
  - `using namespace std;`
- 简单的 if 语句
- 关系比较运算（>, >=, <, <=, ==, !=）
- 优先级与结合律

练习：用 C++ 做比较

小练习

请尝试用 C++ 写程序，输入一元二次方程 $(a \neq 0)$

$$ax^2 + bx + c = 0$$

的二次、一次、常数项系数，输出其有几个实数解。

拓展：可尝试输出具体的解。需要用到`<cmath>`头文件中的`std::sqrt`函数[a]开根号。

[a] https://en.cppreference.com/cpp/numeric/math/sqrt

练习：用 C++ 做比较

```cpp
// example.cpp
#include <iostream>
#include <cmath>
using namespace std;
int main() {
    cout << "Enter a, b, c in order for ax^2+bx+c=0: ";
    float a, b, c;
    cin >> a >> b >> c;
    float delta = b * b - 4 * a * c;
    if (a == 0) {
        cerr << "Invalid input!" << endl;
        return -1;
    }
    if (delta < 0) cout << "There is no real root." << endl;
    if (delta == 0) cout << "There is one real root x = "
                         << - b / (2 * a) << endl;
    if (delta > 0) {
        cout << "There are two real roots: " << endl;
        cout << "x1 = " << (-b - sqrt(delta)) / (2 * a) << endl;
        cout << "x2 = " << (-b + sqrt(delta)) / (2 * a) << endl;
    }
} // end function main
```

总结

- 熟悉基本数据类型✓
  - `bool`, `char`, `short`, `int`, `long`, `float`, `double`; `unsigned`
- 运用运算符、了解优先级和结合律✓（按优先级顺序）
  - 圆括号：最高优先级
  - 算术运算符 (+, −, ∗, /, %)：从左向右，乘除模**优先于**加减
  - 流输入输出 (>>, <<)：从左向右
  - 关系比较 (>, >=, <, <=, ==, !=)：从左向右，比较**优先于**等/不等
  - 赋值 (=)：**从右向左**
- 能够运用 C++ 编写简单的计算机程序✓
  - 注释、空行 (`// comment`, `/* comment */`)
  - 预处理、头文件 (`#include <iostream>`)
  - using 声明 (`using namespace std;`)
  - main 函数 (`int main() {}`)
  - 声明变量 (`int x = 0;`)
  - 输入与输出 (`std::cin >>`, `std::cout <<`)
- 能够编写简单的判断语句✓
  - `if (x == 1) {}`